CAP 4611 Assignment 2

Cameron D Lewis

UCFID 5593788

Section 01

09/14/25

1. K Nearest Neighbors

```
distances_squared = euclidean_dist_squared(X_hat, self.X)

for i in range(n_test):
    # Get distances
    distances_i = distances_squared[i, :]

    # Find indices of k nearest neighbors
    nearest_indices = np.argsort(distances_i)[:self.k]

    # Get labels of k nearest neighbors
    nearest_labels = self.y[nearest_indices]

    # Use mode to find most common label
    predictions[i] = utils.mode(nearest_labels)

return predictions
```

a. KNN Results:

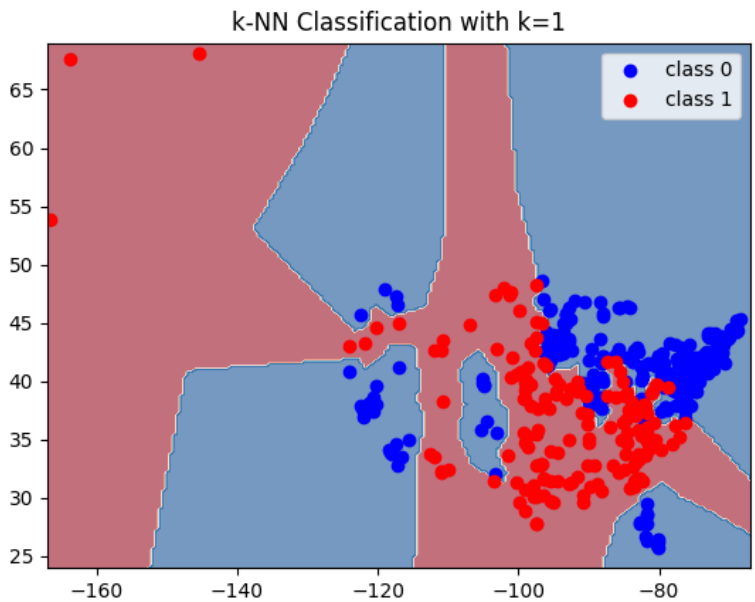
k=1 Training error = 0.000 Test error = 0.065

k=3 Training error = 0.028 Test error = 0.066

k=10 Training error = 0.058 Test error = 0.085

Decision Tree comparison:

Decision Tree: Training error = 0.000, Test error = 0.078Plot



b.

c. When k=1, each point is classified based on its single nearest neighbor which is itself resulting in a 0% training error

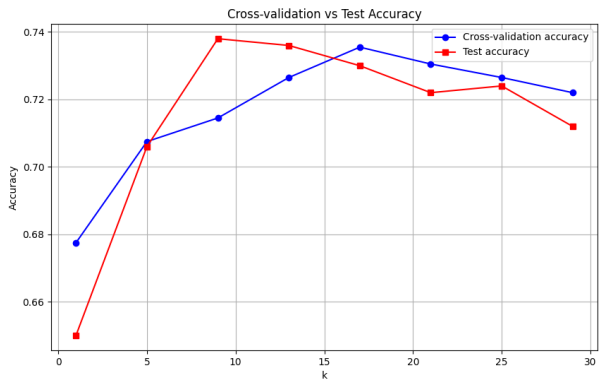
d. Use cross validation

2. Picking K in KNN

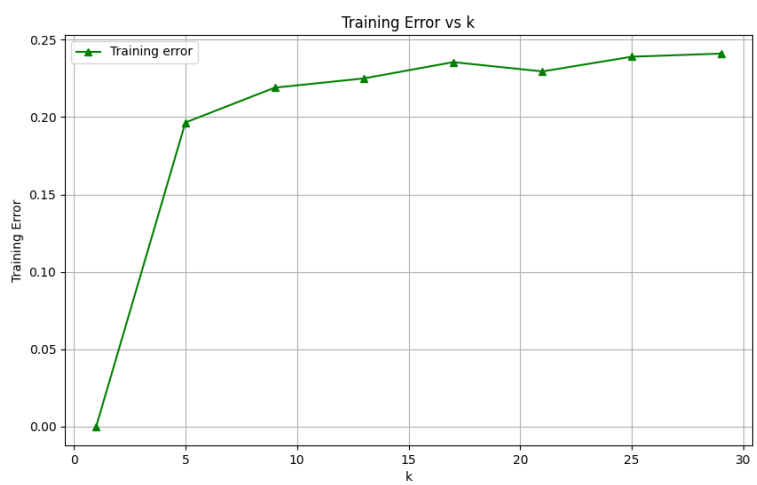
```
n = X.shape[0]

# 10-fold cross-validation
fold_size = n // 10
cv_accs = []
test_accs = []
train_accs = []

for k in ks:
    # Cross-validation accuracies
    fold_accs = []

    for fold in range(10):
        # Create mask for this fold
        mask = np.ones(n, dtype=bool)
        start_idx = fold * fold_size
        if fold == 9:
            end_idx = n
        else:
            end_idx = (fold + 1) * fold_size

        # Set validation indices to False
        mask[start_idx:end_idx] = False

        # Split data
        X_train_fold = X[mask]
        y_train_fold = y[mask]
```

```
        X_val_fold = X[~mask]
        y_val_fold = y[~mask]

        # Train and evaluate
        model = KNN(k=k)
        model.fit(X_train_fold, y_train_fold)
        y_pred_val = model.predict(X_val_fold)
        fold_accuracy = np.mean(y_pred_val == y_val_fold)
        fold_accs.append(fold_accuracy)

    # Store mean accuracy
    cv_accs.append(np.mean(fold_accs))

    # compute test accuracy
    model_full = KNN(k=k)
    model_full.fit(X, y)

    # Training accuracy
    y_pred_train = model_full.predict(X)
    train_accuracy = np.mean(y_pred_train == y)
    train_accs.append(train_accuracy)

    # Test accuracy
    y_pred_test = model_full.predict(X_test)
    test_accuracy = np.mean(y_pred_test == y_test)
    test_accs.append(test_accuracy)

# Plot cross-validation and test accuracies
plt.figure(figsize=(10, 6))
plt.plot(ks, cv_accs, 'b-o', label='Cross-validation
accuracy')
plt.plot(ks, test_accs, 'r-s', label='Test accuracy')
plt.xlabel('k')
plt.ylabel('Accuracy')
plt.title('Cross-validation vs Test Accuracy')
plt.legend()
plt.grid(True)
plt.savefig("../figs/cv_vs_test_accuracy.png")
print("CV vs Test accuracy plot saved as
../figs/cv_vs_test_accuracy.png")

# Plot training error
plt.figure(figsize=(10, 6))
train_errors = [1 - acc for acc in train_accs]
plt.plot(ks, train_errors, 'g-^', label='Training error')
plt.xlabel('k')
plt.ylabel('Training Error')
plt.title('Training Error vs k')
plt.legend()
plt.grid(True)
plt.savefig("../figs/training_error_vs_k.png")
print("Training error plot saved as
../figs/training_error_vs_k.png")

# Find best k values
best_cv_k = ks[np.argmax(cv_accs)]
best_test_k = ks[np.argmax(test_accs)]
```

```
print(f"\nResults:")
print(f"Best k according to cross-validation: {best_cv_k} (CV
accuracy: {max(cv_accs):.3f})")
print(f"Best k according to test accuracy: {best_test_k}
(Test accuracy: {max(test_accs):.3f})")
print(f"CV-chosen k test accuracy:
{test_accs[ks.index(best_cv_k)]:.3f}")
```



The cross-validation would most likely choose 17 as it has the highest accuracy and it would do good because the CV chosen k is similar in accuracy to the best k chosen by test accuracy



The k with the best training error is k=1 0% but that is skewed and would likely do poorly on test data

3. Naïve Bayes
    a. 3.1

i. .6 and .4
ii. 1/6, 5/6,2/6, ¾, ¼, ¾,
iii. Pr(= 1 | spam) * Pr(lottery = 1 | spam)* Pr(Venmo = 0 | spam) = .0463
Pr(= 1 | notspam) * Pr(lottery = 1 | notspam)* Pr(Venmo = 0 | notspam) = .1406
.0436<.1406 so it would most likely not be spam
iv. [0 0 0 spam
1 1 1 not spam insert these extra rows into the original
0 0 0 spam
1 1 1] not spam

b. 3.2
i. Word at column 73 (index 72): question
ii. Words present in example 803: ['case', 'children', 'health', 'help', 'problem', 'program']
iii. Example 803 belongs to the newsgroup: talk.*

c. 3.3

```
p_xy = np.zeros((d, k))

for c in range(k):
    # Find examples belonging to class c
    class_mask = (y == c)
    n_c = np.sum(class_mask)

    if n_c > 0:
        for j in range(d):
            feature_count = np.sum(X[class_mask, j])
            p_xy[j, c] = feature_count / n_c

self.p_y = p_y
self.p_xy = p_xy
```

i. Naive Bayes training error: 0.200
Naive Bayes validation error: 0.188

d. 3.4

```
p_xy = np.zeros((d, k))

for c in range(k):
    # Find examples belonging to class c
    class_mask = (y == c)
    n_c = np.sum(class_mask)

    for j in range(d):
        feature_count = np.sum(X[class_mask, j])
        # Apply Laplace smoothing
        p_xy[j, c] = (feature_count + self.beta) /
```

```
(n_c + 2 * self.beta)

self.p_y = p_y
self.p_xy = p_xy
```

i.

ii. Yes some probability was 0 before and after doing B=1 none of them are zero anymore

iii. Yes because as the smoothing becomes too intensive the outcomes become equally as likely

e. 3.5

i. O(tkd)

4. Random Forests

a. Because the random tree only looks at a subset of features at each split

b. Splits stop when no further improvements are made

```
def __init__(self, num_trees, max_depth):
    self.num_trees = num_trees
    self.max_depth = max_depth
    self.trees = []


def fit(self, X, y):
    self.trees = []

    for i in range(self.num_trees):
        # Create a new RandomTree
        tree = RandomTree(max_depth=self.max_depth)

        # Fit the tree to the data
        tree.fit(X, y)

        # Add to our forest
        self.trees.append(tree)

def predict(self, X_pred):
    n = X_pred.shape[0]

    # Get predictions from all trees
    tree_predictions = np.zeros((self.num_trees, n))

    for i, tree in enumerate(self.trees):
        tree_predictions[i] = tree.predict(X_pred)

    # For each example, take the mode across all trees
    predictions = np.zeros(n)
    for i in range(n):
        predictions[i] = utils.mode(tree_predictions[:,
i])
```

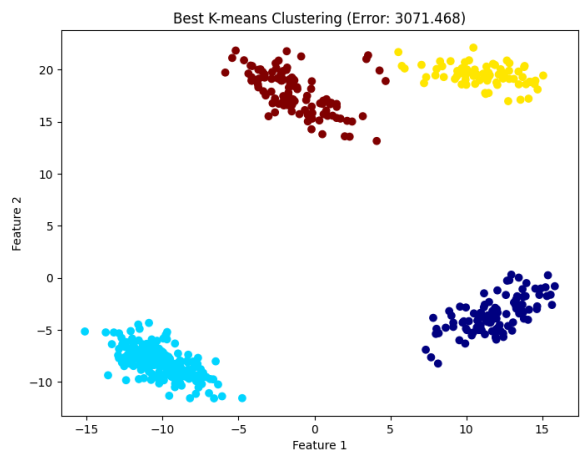
```
    return predictions
```

c. Training error: 0.000
   Testing error: 0.170
d. Random forests typically have 0 training error because with enough trees the ensemble can memorize the training data

5. Clustering
   a. 5.1

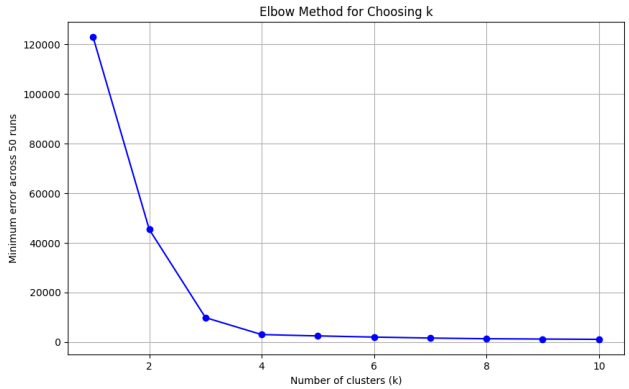
```
total_error = 0
for i in range(X.shape[0]):
    # Get the assigned cluster for example i
    cluster = y[i]
    # Compute squared distance from example i to its
assigned mean
    diff = X[i] - means[cluster]
    squared_dist = np.sum(diff ** 2)
    total_error += squared_dist

return total_error
```

      i. Error decreases with each iteration



      ii. Lowest error obtained: 3071.468

   b. 5.2
      i. Error decreases as k increases reaching 0 when k=n
      ii. No because clustering is unsupervised



iii.

iv. I would choose k=3 since the slope is quite large and the data wouldn't be as black and white as say k=2

6. Very Short Answer Questions
    a. If emails come from the same sender, it could violate the independence assumption
    b. It usually leads to overfitting and poor generalization of new data
    c. As n increases, optimization bias decreases
    d. Advantage, more training data per fold Disadvantage, higher cost
    e. P(xi) is the same for all classes
    f.
        i. parameter
        ii. parameter
        iii. hyperparameter
    g. labeled training data vs patterns in unlabeled data
    h. No it can be non convex